

WEEK3 Relational Design Theory

CSC404 — Relational Design Theory

Student Notes | Functional Dependencies · BCNF · Multivalued Dependencies · 4NF

Table of Contents

1. Why Does Schema Design Matter?
 2. Design Anomalies
 3. Functional Dependencies (FDs)
 4. Closure of Attributes
 5. Boyce-Codd Normal Form (BCNF)
 6. Multivalued Dependencies (MVDs)
 7. Fourth Normal Form (4NF)
 8. Shortcomings of BCNF and 4NF
 9. Hierarchy of Normal Forms
 10. Quick Reference Summary
-

1. Why Does Schema Design Matter?

When designing a database, many schema layouts are possible — but most of them are bad. The goal of **Relational Design Theory** is to give us a *principled, automatic* way to arrive at a good schema.

The two main toolkits are:

Tool	Detects	Leads to
Functional Dependencies (FDs)	Value-to-value constraints	Boyce-Codd Normal Form (BCNF)
Multivalued Dependencies (MVDs)	Independent repeating facts	Fourth Normal Form (4NF)

The general approach is called **Design by Decomposition**:

Start with one big "mega" relation containing all attributes, then systematically break it down using dependency rules until no anomalies remain.

2. Design Anomalies

The Problem: A Bad Schema

Consider student Ann (SSN = 123) who:

- Attended **PAHS** and **GHS** (both in Palo Alto)
- Plays **tennis** and **trumpet**
- Applies to **Stanford**, **Berkeley**, and **MIT**

A naive designer might create one flat relation:

Apply(SSN, sName, cName, HSname, HScity, hobby)

This requires **1 student × 2 high schools × 2 hobbies × 3 colleges = 12 tuples**.

- Ann's SSN–name pair is stored **12 times**
- The fact "Ann attends PAHS" is stored **6 times**

Redundancy = every piece of information captured many more times than necessary.

Three Anomalies That Follow

① *Redundancy Anomaly*

Each fact is stored multiple times. This wastes storage and creates inconsistency risk.

② *Update Anomaly*

To change one fact (e.g., Ann's name), you must update **all 12 copies**. If only 3 of 12 are updated, the database becomes inconsistent.

③ *Deletion Anomaly*

Deleting one piece of data accidentally removes other data. For example, removing all tuples where hobby = 'surfing' might cause Ann to disappear from the database entirely if surfing was her only hobby in those records.

A Better Design — Decompose It

Split the mega-relation into smaller, purpose-built relations:

Student(SSN, sName)

Apply(SSN, cName)

HighSchool(SSN, HSname, HScity)

Hobbies(SSN, hobby)

Benefits:

- Each fact is stored exactly **once** — no redundancy
- No update or deletion anomalies
- The original mega-relation can be **reconstructed by joining** these tables

The best design depends on what the data represents in the real world.

3. Functional Dependencies (FDs)

Definition

A **functional dependency** $A \rightarrow B$ means:

Any two tuples (rows) that agree on the value(s) of A must also agree on the value(s) of B.

Formally: If $t[A] = u[A]$, then $t[B] = u[B]$ for all tuples t, u in relation R .

Think of it like a mathematical function: knowing A tells you exactly what B is.

FDs are **not** derived from data — they come from your knowledge of the real world. Every instance of the relation must satisfy every stated FD.

Examples

Using Student(SSN, sName, address, HScode, HSname, HScity, GPA, priority):

FD	Meaning
$SSN \rightarrow sName$	Each SSN belongs to exactly one student
$SSN \rightarrow address$	One address per student
$HScode \rightarrow HSname, HScity$	A school code uniquely

FD	Meaning
	identifies a school
GPA $\rightarrow$ priority	Priority is determined purely by GPA

Multiple attributes are allowed on either side: $A_1, A_2 \rightarrow B_1, B_2$.

FDs generalise the concept of keys — a key is just an attribute (or set of attributes) that functionally determines *all* other attributes.

Types of FDs

Type	Definition	Example
Trivial	$A \rightarrow B$ where $B \subseteq A$ (RHS already inside LHS)	$\{SSN, sName\} \rightarrow sName$
Non-trivial	$A \rightarrow B$ where at least some of B is not in A	$\{SSN, sName\} \rightarrow \{sName, GPA\}$
Completely Non-trivial	$A \rightarrow B$ where $A \cap B = \emptyset$ (no overlap at all)	$SSN \rightarrow GPA$

For schema design, we primarily care about **completely non-trivial FDs** — these are the ones that tell us something genuinely new.

Rules for FDs

These rules let us derive new FDs from existing ones:

Splitting Rule

$A \rightarrow B_1, B_2, \dots, B_n$ implies $A \rightarrow B_1, A \rightarrow B_2, \dots, A \rightarrow B_n$

You can split the **right-hand side** (RHS) into individual FDs.

Splitting does NOT work on the left-hand side (LHS)!

$SSN, HScore \rightarrow GPA$ does NOT imply $SSN \rightarrow GPA$ or $HScore \rightarrow GPA$.

Combining Rule

If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow B, C$.

Merge multiple FDs that share the same LHS.

Trivial-Dependency Rule

- $A \rightarrow B$ implies $A \rightarrow A, B$ (add LHS attributes to RHS)
- $A \rightarrow A, B$ implies $A \rightarrow B$ (remove LHS attributes from RHS)

Transitive Rule

If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

Example: $SSN \rightarrow GPA$ and $GPA \rightarrow \text{priority}$ together imply $SSN \rightarrow \text{priority}$.

These rules are all formally provable. Together with reflexivity and augmentation, they form **Armstrong's Axioms** — a complete set of inference rules for FDs.

4. Closure of Attributes

Definition

The **closure** of an attribute set A (written A^+) is the set of **all** attributes that are functionally determined by A , given a set of FDs.

In other words: starting from A , what can you figure out?

Algorithm

1. Start with the set $\{A\}$
2. Repeat until no change:
If there exists an FD $X \rightarrow Y$ such that $X \subseteq \text{current set}$:
Add Y to the current set
3. The final set is A^+

Example

Given Student(SSN, sName, address, HScore, HSname, HScity, GPA, priority) with:

- $SSN \rightarrow \text{sName, address, GPA}$
- $GPA \rightarrow \text{priority}$
- $HScore \rightarrow \text{HSname, HScity}$

Find the closure of {SSN, HScore}:

Step	Action	Current Set
Start	—	{SSN, HScore}

Step	Action	Current Set
Apply SSN $\rightarrow$ sName, address, GPA	Add sName, address, GPA	{SSN, HScore, sName, address, GPA}
Apply GPA $\rightarrow$ priority	Add priority	{SSN, HScore, sName, address, GPA, priority}
Apply HScore $\rightarrow$ HSname, HScity	Add HSname, HScity	{SSN, HScore, sName, address, GPA, priority, HSname, HScity}

Since $\{\text{SSN, HScore}\}^+ = \text{all attributes}$, **{SSN, HScore} is a key** for this relation.

Finding Keys via Closure

- If $A^+ = \text{all attributes of } R \rightarrow \mathbf{A \text{ is a key for } R}$
- To find **all keys**: enumerate every subset of attributes and compute its closure
 - o If closure = all attributes $\rightarrow$ that subset is a key
 - o Optimisation: once A is a key, every superset of A is also a key (superkey), so skip those

5. Boyce-Codd Normal Form (BCNF)

Definition

A relation R is in BCNF if, for every non-trivial FD $A \rightarrow B$, A is a key of R.

Equivalently: the only non-trivial FDs allowed are those where the **left-hand side is a key**.

Why This Eliminates Anomalies

If $A \rightarrow B$ holds but A is *not* a key, then multiple tuples can share the same A value. Each of those tuples must also share the same B value — meaning the same piece of information (B) is stored over and over again. BCNF prevents this by requiring A to **uniquely identify every tuple**.

Example: BCNF Violation

Relation: Apply(SSN, sName, cName) with FD $\text{SSN} \rightarrow \text{sName}$

- The **key** is {SSN, cName} (a student can apply to multiple colleges)

- SSN is **not** the key
- Therefore, for every college Ann applies to, her name is stored again →
BCNF VIOLATION

Fix: Decompose into:

Student(SSN, sName) ← SSN is key ✓

Apply(SSN, cName) ← {SSN, cName} is key ✓

BCNF Decomposition Algorithm

INPUT: Relation R with its FDs

OUTPUT: A lossless decomposition into BCNF relations

1. Compute key(s) for R using FD closure
 2. Repeat until all relations are in BCNF:
 - a. Pick any relation R' with a violating FD: $A \rightarrow B$ (A is not a key)
 - b. Decompose R' into:
 - $R_1 = A \cup B$ ← attributes IN the FD
 - $R_2 = (R' - B) \cup A$ ← LHS + all remaining attributes
 - c. Compute FDs and keys for R_1 and R_2
 3. Return the set of decomposed relations
- **Lossless join guaranteed:** $R = R_1 \bowtie R_2$ always holds after a BCNF decomposition.
 - **Non-deterministic:** choosing different violating FDs may yield different (but equally valid) BCNF schemas.
-

Worked Example: BCNF Decomposition

Start:

Student(SSN, sName, address, HScode, HSname, HScity, GPA, priority)

FDs:

$SSN \rightarrow sName, address, GPA$

$GPA \rightarrow priority$

$HScode \rightarrow HSname, HScity$

Key: {SSN, HScode} — all three FDs violate BCNF

Step 1: Use HScore $\rightarrow$ HSname, HScity

S1(HScore, HSname, HScity) $\leftarrow$ HScore is key ✓ [BCNF]

S2(SSN, sName, address, HScore, GPA, priority) $\leftarrow$ still violates

Step 2: Use GPA $\rightarrow$ priority on S2

S3(GPA, priority) $\leftarrow$ GPA is key ✓ [BCNF]

S4(SSN, sName, address, HScore, GPA) $\leftarrow$ still violates

Step 3: Use SSN $\rightarrow$ sName, address, GPA on S4

S5(SSN, sName, address, GPA) $\leftarrow$ SSN is key ✓ [BCNF]

S6(SSN, HScore) $\leftarrow$ {SSN, HScore} is key ✓ [BCNF]

Final schema: S1, S3, S5, S6 — each relation is in BCNF, no anomalies!

6. Multivalued Dependencies (MVDs)

Why BCNF is Sometimes Not Enough

Consider: Apply(SSN, cName, hobby)

Assume students can apply to multiple colleges and can have multiple hobbies, and that **colleges and hobbies are completely independent of each other**.

- FDs? $\rightarrow$ None (SSN alone doesn't determine cName or hobby)
- Key? $\rightarrow$ {SSN, cName, hobby} (all three attributes)
- In BCNF? $\rightarrow$ **Yes** (no non-trivial FDs to violate it)
- Good design? $\rightarrow$ **No!**

The problem: a student with 5 colleges and 6 hobbies needs $5 \times 6 = 30$ tuples, when we only need $5 + 6 = 11$. This **multiplicative redundancy** is invisible to FDs.

BCNF handles FD-based redundancy. 4NF handles independence-based redundancy.

Definition

A **multivalued dependency** $A \twoheadrightarrow B$ means:

For a given value of A, the set of B values associated with it is **independent** of all other attributes.

Formal definition: If tuples t and u agree on A, then there must exist a tuple v such that:

- $v[A] = t[A] = u[A]$
- $v[B] = t[B]$ (takes B-value from t)
- $v[\text{rest}] = u[\text{rest}]$ (takes remaining values from u)

In plain English: for any A value, **all combinations** of its B values and its "other" values must appear in the relation.

Example

Apply(SSN, cName, hobby) with $\text{SSN} \twoheadrightarrow \text{cName}$:

If these two tuples exist:

SSN	cName	hobby
123	Stanford	tennis
123	Berkeley	trumpet

Then the following **must also exist**:

SSN	cName	hobby
123	Stanford	trumpet
123	Berkeley	tennis

Note: $\text{SSN} \twoheadrightarrow \text{cName}$ also implies $\text{SSN} \twoheadrightarrow \text{hobby}$ by the **complementation rule** — if one set of values is independent of the others, so is the remaining set.

Rules for MVDs

Rule	Statement
FD-is-an-MVD	If $A \rightarrow B$, then $A \twoheadrightarrow B$. Every FD is also an MVD (special case). This means 4NF is strictly stronger than BCNF .
Intersection Rule	If $A \twoheadrightarrow B$ and $A \twoheadrightarrow C$, then $A \twoheadrightarrow (B \cap C)$

Rule	Statement
Transitive Rule	If $A \twoheadrightarrow B$ and $B \twoheadrightarrow C$, then $A \twoheadrightarrow (C - B)$

Trivial MVD: $A \twoheadrightarrow B$ is trivial if $B \subseteq A$ or $A \cup B = \text{all attributes}$.

Non-trivial MVD: everything else — these are the ones we care about for design.

The **splitting rule applies to FDs but NOT always to MVDs**.

7. Fourth Normal Form (4NF)

Definition

A relation R is in 4NF if, for every non-trivial MVD $A \twoheadrightarrow B$, A is a key of R.

- 4NF **implies** BCNF (since every FD is also an MVD, every BCNF check is also a 4NF check)
- 4NF is **stronger** than BCNF — fewer relations satisfy it

Why It Eliminates Multiplicative Redundancy

If $A \twoheadrightarrow B$ and A is **not** a key, then multiple A values exist in the relation, forcing all $B \times (\text{other attributes})$ combinations to appear → **exponential blowup**.

If A **is** a key, there is only one tuple per A value, so no blowup is possible.

Example: 4NF Fix

Apply(SSN, cName, hobby) with $\text{SSN} \twoheadrightarrow \text{cName}$:

- SSN is not a key → **4NF VIOLATION**

Fix: Decompose into:

Apply(SSN, cName) ← {SSN, cName} is key ✓ [4NF]

Hobbies(SSN, hobby) ← {SSN, hobby} is key ✓ [4NF]

Now a student with 5 colleges and 6 hobbies needs only $5 + 6 = 11$ tuples, not 30.

4NF Decomposition Algorithm

INPUT: Relation R with FDs and MVDs

OUTPUT: A lossless decomposition into 4NF relations

1. Compute keys for R using FDs (NOT MVDs — MVDs don't define keys)
2. Repeat until all relations are in 4NF:
 - a. Pick any R' with a nontrivial violating MVD: $A \twoheadrightarrow B$ (A is not a key)
 - b. Decompose R' into:
 $R_1 = A \cup B \leftarrow$ attributes IN the MVD
 $R_2 = (R' - B) \cup A \leftarrow$ LHS + all remaining attributes
 - c. Compute FDs, MVDs, and keys for R_1 and R_2
3. Return the set of decomposed relations

The structure is identical to the BCNF algorithm, except we use **MVD violations** instead of FD violations.

Worked Example: 4NF Decomposition

Start:

Apply(SSN, cName, date, major, hobby)

FD: {SSN, cName} $\rightarrow$ date (a student applies to each college on one date)

MVD: {SSN, cName, date} $\twoheadrightarrow$ major (majors and hobbies are independent)

Key: all 5 attributes $\rightarrow$ both FD and MVD violate 4NF

Step 1: Decompose on MVD {SSN, cName, date} $\twoheadrightarrow$ major

A1(SSN, cName, date, major) $\leftarrow$ still violates (FD {SSN,cName} $\rightarrow$ date)

A2(SSN, cName, date, hobby) $\leftarrow$ still violates (same FD)

Step 2: FD {SSN, cName} $\rightarrow$ date violates 4NF in both A1 and A2

From A1: A3(SSN, cName, date) A4(SSN, cName, major)

From A2: A3(SSN, cName, date) A5(SSN, cName, hobby) $\leftarrow$ A3 is the same!

Final schema:

A3(SSN, cName, date) $\leftarrow$ {SSN, cName} is key ✓ [4NF]

A4(SSN, cName, major) $\leftarrow$ {SSN, cName, major} is key ✓ [4NF]

A5(SSN, cName, hobby) $\leftarrow$ {SSN, cName, hobby} is key ✓ [4NF]

All three relations are in 4NF — independent facts are stored separately!

8. Shortcomings of BCNF and 4NF

Good theory, but real-world trade-offs exist.

Problem 1 — Lost Dependencies

Decomposition can **split an FD across different relations**, making it impossible to enforce without a join.

Example: Apply(SSN, cName, date, major) with $\text{date} \rightarrow \text{cName}$

BCNF decomposes into:

A1(cName, date)

A2(SSN, date, major)

Now checking the dependency $\{\text{SSN}, \text{cName}\} \rightarrow \text{major}$ requires **joining A1 and A2** every time you want to verify it — expensive!

Problem 2 — Query Workload

If most queries already need **all attributes**, then decomposing into many small tables forces every query to perform costly joins.

Example: Student(SSN, sName, SAT, WAEC) — if all queries use all four columns, decomposing into 3 tables means every query must do 2 joins.

In such cases, **intentional denormalisation** (keeping a schema that violates BCNF/4NF) may actually perform better.

Problem 3 — Over-decomposition

It's technically possible to decompose a perfectly fine table into many tiny single-attribute tables that are all in BCNF/4NF but are completely impractical to use.

Third Normal Form (3NF) is a weaker but dependency-preserving alternative to BCNF. It allows some redundancy in exchange for keeping all FDs enforceable within a single relation.

9. Hierarchy of Normal Forms

All relational schemas

- └── 1NF: atomic values in every cell (all standard tables satisfy this)
- └── 2NF: no partial dependencies on a composite key
- └── 3NF: no transitive dependencies — slightly weaker than BCNF
- └── BCNF: every non-trivial FD has a key on LHS
- └── 4NF: every non-trivial MVD has a key on LHS

Subset relationship: $4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$

Normal Form	Handles	Use When
BCNF	FD-based redundancy	Most standard schema design
4NF	FD + MVD-based redundancy	Independent multi-valued facts exist
3NF	FD-based redundancy (weaker)	Dependency preservation is essential
1NF / 2NF	Historical stepping stones	Rarely discussed in modern design

10. Quick Reference Summary

FD vs MVD

	Functional Dependency (FD)	Multivalued Dependency (MVD)
–		
Notation	$A \rightarrow B$	$A \twoheadrightarrow B$
Meaning	Same A $\rightarrow$ same B (one value)	Same A $\rightarrow$ all combinations of B $\times$ rest
Trivial when	$B \subseteq A$	$B \subseteq A$ or $A \cup B = \text{all attributes}$
Leads to	BCNF	4NF
Splitting on RHS	✓ Yes	✗ Not always
Relationship	FD is a special case of MVD	MVD is the more general concept

BCNF vs 4NF

	BCNF	4NF
Based on	FDs only	FDs + MVDs
Rule	Every non-trivial FD must have key on LHS	Every non-trivial MVD must have key on LHS
Eliminates	FD-based redundancy	FD + MVD-based (multiplicative) redundancy
Strength	Weaker	Stronger ($4NF \subset BCNF$)
Weakness	May lose FD constraints	Even more join overhead

Design by Decomposition — Full Process

1. Identify all relevant attributes
2. Create a mega-relation with all attributes
3. Specify FDs (and MVDs if needed) from real-world knowledge
4. Compute keys using the closure algorithm
5. Test each FD/MVD for BCNF/4NF violations
6. Decompose iteratively using the appropriate algorithm
7. Verify:
 - ✓ Lossless join property holds
 - ✓ All relations are in the target normal form
 - ✓ Query workload is manageable (consider denormalisation if needed)
8. Repeat from step 5 until all relations satisfy the normal form

Key Definitions at a Glance

Term	Definition
Functional Dependency $A \rightarrow B$	Knowing A tells you exactly what B is
Closure A^+	All attributes functionally determined by A

Term	Definition
Key	An attribute set whose closure = all attributes
BCNF	Every non-trivial FD has a key on the left
Multivalued Dependency $A \twoheadrightarrow B$	A determines a <i>set</i> of B values, independent of other attributes
4NF	Every non-trivial MVD has a key on the left
Lossless decomposition	The original relation can be fully recovered by joining the decomposed parts
Denormalisation	Intentionally violating a normal form for performance reasons

Notes compiled from: CSC404 — Database Management Systems, Relational Design Theory (Iheagwara). Based on Widom (2012).